

# Sistema Solar — Guia de Construção Passo a Passo

---

**Projeto:** Sistema de Gestão de Vendas, Estoque e Financeiro

**Stack:** Python · FastAPI · SQLAlchemy · Pydantic · SQLite · Streamlit

**Nível:** Iniciante/Intermediário com background em VBA/Excel

**Objetivo:** Referência completa para reconstruir o sistema do zero

## Índice

---

1. [Conceitos Fundamentais](#)
2. [Estrutura de Arquivos](#)
3. [Passo 1 — database.py](#)
4. [Passo 2 — enums.py](#)
5. [Passo 3 — Model Cliente](#)
6. [Passo 4 — Schema Cliente](#)
7. [Passo 5 — Route Cliente](#)
8. [Passo 6 — Model Produto](#)
9. [Passo 7 — Model Venda + ItemVenda](#)
10. [Passo 8 — Schema e Route Venda](#)
11. [Passo 9 — Model Colaborador](#)
12. [Passo 10 — Model EstoqueMovimentacao](#)
13. [Passo 11 — Model Financeiro](#)
14. [Passo 12 — Aprovação Completa \(transação atômica\)](#)
15. [Passo 13 — Model PrestacaoContas](#)
16. [Passo 14 — main.py final](#)
17. [Passo 15 — Dashboard endpoint](#)
18. [Passo 16 — Frontend Streamlit](#)
19. [Erros Comuns](#)
20. [Próximos Passos](#)

## Conceitos Fundamentais

---

### O que é uma Classe?

Uma classe é um **molde** — define a estrutura de um objeto, mas não é o objeto em si.

**Analogia Excel:** a linha de cabeçalho de uma planilha define a estrutura (colunas). Cada linha de dados é uma *instância* dessa estrutura.

```
class Cliente:          # → o cabeçalho (molde)
    nome: str
    cpf: str

c1 = Cliente()          # → linha 1 de dados (instância)
c2 = Cliente()          # → linha 2 de dados (objeto diferente, mesmo molde)
```

Quando o SQLAlchemy herda a classe de `Base`, esse molde passa a representar uma **tabela no banco de dados**. Cada instância é uma linha dessa tabela.

## Arquitetura do projeto

| Arquivo/Pasta            | Analogia VBA         | Responsabilidade                                    |
|--------------------------|----------------------|-----------------------------------------------------|
| <code>enums.py</code>    | Constantes globais   | Valores fixos reutilizados em todo o sistema        |
| <code>models/</code>     | Definição de tabelas | Diz ao banco quais colunas existem                  |
| <code>schemas/</code>    | UserForm (validação) | Valida o que entra e formata o que sai da API       |
| <code>routes/</code>     | Botão + Sub/Function | Recebe requisição, executa lógica, devolve resposta |
| <code>database.py</code> | ADODB.Connection     | Configura a conexão com o banco de dados            |
| <code>main.py</code>     | Módulo principal     | Amarra tudo — ponto de entrada da aplicação         |

## Fluxo completo de uma requisição

Usuário clica em **"Salvar"** no Streamlit

- Streamlit chama `api_client.criar_cliente(dados)`
- `api_client` faz POST `http://localhost:8000/clientes/`
- FastAPI recebe **a** requisição
- Pydantic valida os dados (`ClienteCreate`) ← rejeita se inválido
- Route `criar_cliente()` executa **a** lógica
- SQLAlchemy faz INSERT na tabela `clientes`
- Pydantic formata **a** resposta (`ClienteResponse`)
- FastAPI devolve JSON com status 201
- Streamlit exibe mensagem de sucesso

## Por que separar Model e Schema?

**Model** (SQLAlchemy) = o que existe no banco de dados

**Schema** (Pydantic) = o contrato com quem usa a API

Exemplo prático: o campo `id` é gerado pelo banco automaticamente. Se você usar o model diretamente como formulário de entrada, o usuário teria que enviar um `id` que ainda não existe. O Schema `ClienteCreate` não inclui `id` — o Schema `ClienteResponse` inclui.

| Campo         | ClienteCreate    | ClienteResponse |
|---------------|------------------|-----------------|
| nome          | ✓ obrigatório    | ✓ retornado     |
| cpf_cnpj      | ✓ obrigatório    | ✓ retornado     |
| id            | ✗ não enviado    | ✓ retornado     |
| status        | ✗ sistema define | ✓ retornado     |
| data_cadastro | ✗ sistema define | ✓ retornado     |

## Por que `get_db()` existe?

Em sistemas web, várias pessoas acessam ao mesmo tempo. Uma conexão global causaria conflito entre requisições simultâneas. O `get_db()` garante que **cada requisição tem sua própria sessão**, que abre e fecha automaticamente — mesmo que ocorra um erro.

```
def get_db():
    db = SessionLocal() # abre sessão
    try:
        yield db         # entrega para o endpoint usar
    finally:
        db.close()      # fecha SEMPRE, mesmo se der erro
```

O `yield` é o que torna isso possível: o código antes do `yield` executa antes do endpoint, e o código depois (no `finally`) executa depois — independente de sucesso ou erro.

## Por que `db.flush()` antes de `db.commit()` ?

Ao criar uma Venda com Itens, você tem um problema de ordem:

1. Cria a Venda → banco gera `id = 7` (mas só após gravar)
2. Cria ItemVenda → precisa de `venda_id = 7`

`db.flush()` envia o SQL ao banco e obtém o ID gerado, **mas não finaliza a transação**. Se algo falhar nos itens, tudo (venda + itens) é revertido junto.

`db.commit()` finaliza a transação — a partir desse momento os dados são permanentes.

## O que é uma transação atômica?

"Atômica" significa **tudo ou nada**. Na aprovação de uma venda, 5 operações acontecem:

1. Mudar status da venda
2. Baixar estoque de cada item
3. Criar receita de equipamentos
4. Criar receita de instalação
5. Criar prestação de contas

Se a operação 4 falhar, as operações 1, 2 e 3 são desfeitas automaticamente. O banco nunca fica em estado inconsistente (venda aprovada mas sem lançamento financeiro, por exemplo).

Isso funciona porque todas as operações usam a **mesma sessão db**, e o `db.commit()` só ocorre no final.

## @property — campo calculado

Equivalente a uma fórmula em célula do Excel. Não é salvo no banco, calculado na hora que você acessa:

```
@property
def margem_pct(self):
    return (self.preco_venda - self.custo_unitario) / self.preco_venda
```

Quando você escreve `produto.margem_pct`, o Python executa essa função. Para aparecer na resposta da API, basta declarar o campo no schema com `from_attributes=True`.

## Estrutura de Arquivos

```
SOLAR/
├─ app/
│   ├── __init__.py
│   ├── main.py
│   ├── database.py
│   └── enums.py
```

← ponto de entrada, registra todos os routers  
← conexão com banco, Base, get\_db()  
← todos os enums do sistema

```

|   |   | models/
|   |   | |   | __init__.py
|   |   | |   | cliente.py
|   |   | |   | produto.py
|   |   | |   | venda.py
|   |   | |   | item_venda.py
|   |   | |   | colaborador.py
|   |   | |   | estoque.py
|   |   | |   | financeiro.py
|   |   | |   | prestacao_contas.py
|   |   | | schemas/
|   |   | | |   | __init__.py
|   |   | | |   | cliente.py
|   |   | | |   | produto.py
|   |   | | |   | venda.py
|   |   | | |   | colaborador.py
|   |   | | |   | estoque.py
|   |   | | |   | financeiro.py
|   |   | | |   | prestacao_contas.py
|   |   | | routes/
|   |   | | |   | __init__.py
|   |   | | |   | cliente.py
|   |   | | |   | produto.py
|   |   | | |   | venda.py
|   |   | | |   | colaborador.py
|   |   | | |   | estoque.py
|   |   | | |   | financeiro.py
|   |   | | |   | prestacao_contas.py
|   |   | | |   | dashboard.py
|   | streamlit_app/
|   | | app.py
|   | | api_client.py
|   | | | pages/
|   | | | | 01_Clientes.py
|   | | | | 02_Vendas.py
|   | | | | 03_Estoque.py
|   | | | | 04_Financeiro.py
|   | | | | 05_Prestacoes.py
|   | solar.db
|   | requirements.txt
|   | GUIA.md

```

← página inicial (Dashboard)

← todas as chamadas HTTP

← banco SQLite (gerado automaticamente)

**Por que `__init__.py` ?** No Python, para que uma pasta seja tratada como um pacote importável, ela precisa ter esse arquivo. Pode estar vazio — a existência dele é o que importa.

## Passo 1 — `database.py`

**Problema resolvido:** configurar a conexão com o banco e fornecer sessões isoladas por requisição.

```

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase

DATABASE_URL = "sqlite:///./solar.db"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}
)

SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

class Base(DeclarativeBase):
    pass

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

**DATABASE\_URL** : endereço do banco. `sqlite:///./solar.db` = arquivo `solar.db` na pasta atual.

**engine** : o motor de conexão. Equivale ao `ADODB.Connection` do VBA.

**check\_same\_thread=False** : o SQLite normalmente só aceita acesso da thread que o criou. O FastAPI usa múltiplas threads — esse parâmetro libera isso.

**autocommit=False, autoflush=False** : controle manual das transações. Você decide quando confirmar ( `commit` ) ou cancelar ( `rollback` ).

**class Base** : molde mestre. Todo model herda dela, o que registra a tabela no SQLAlchemy.

## Passo 2 — `enums.py`

**Problema resolvido**: evitar duplicação de constantes. Se o mesmo enum for definido em dois arquivos, o Python trata como **tipos completamente diferentes** — mesmo que os valores sejam iguais. Isso causa erros silenciosos na hora de gravar no banco.

**Regra**: defina cada enum uma única vez. Todos os outros arquivos importam daqui.

```

import enum

class StatusCliente(enum.Enum):
    ATIVO = "Ativo"

```

```
INATIVO    = "Inativo"  
PROSPECTO = "Prospecto"  
BLOQUEADO = "Bloqueado"
```

```
class TipoPessoa(enum.Enum):  
    PF = "PF"  
    PJ = "PJ"
```

```
class CategoriaProduto(enum.Enum):  
    PAINEL    = "Painel"  
    INVERSOR  = "Inversor"  
    CABO      = "Cabo"  
    ESTRUTURA = "Estrutura"  
    ACESSORIO = "Acessório"  
    SERVICO   = "Serviço"
```

```
class UnidadeProduto(enum.Enum):  
    UN = "UN"  
    M  = "M"  
    KG = "KG"  
    KWP = "KWP"
```

```
class StatusVenda(enum.Enum):  
    ORCAMENTO    = "Orçamento"  
    NEGOCIACAO   = "Negociação"  
    APROVADO     = "Aprovado"  
    EM_ANDAMENTO = "Em Andamento"  
    CONCLUIDO    = "Concluído"  
    CANCELADO    = "Cancelado"
```

```
class FormaPagamento(enum.Enum):  
    PIX          = "PIX"  
    BOLETO       = "Boleto"  
    DINHEIRO     = "Dinheiro"  
    CARTAO       = "Cartão"  
    FINANCIAMENTO = "Financiamento"
```

```
class TipoContrato(enum.Enum):  
    CLT          = "CLT"  
    AUTONOMO     = "Autônomo"  
    PJ           = "PJ"
```

```
class StatusColaborador(enum.Enum):  
    ATIVO    = "Ativo"  
    INATIVO  = "Inativo"
```

```
class TipoMovimento(enum.Enum):
    ENTRADA = "Entrada"
    SAIDA = "Saída"
    AJUSTE_POSITIVO = "Ajuste+"
    AJUSTE_NEGATIVO = "Ajuste-"

class TipoLancamento(enum.Enum):
    RECEITA = "Receita"
    DESPESA = "Despesa"

class CategoriaFinanceiro(enum.Enum):
    VENDA_EQUIPAMENTOS = "Venda Equipamentos"
    SERVICO_INSTALACAO = "Serviço Instalação"
    MANUTENCAO = "Manutenção"
    COMISSAO = "Comissão"
    FORNECEDOR = "Fornecedor"
    SALARIO = "Salário"
    ALUGUEL = "Aluguel"
    MARKETING = "Marketing"
    IMPOSTO = "Imposto"
    OUTRAS = "Outras"

class StatusPagamento(enum.Enum):
    PENDENTE = "Pendente"
    PAGO = "Pago"
    ATRASADO = "Atrasado"
    CANCELADO = "Cancelado"

class StatusPrestacao(enum.Enum):
    PENDENTE = "Pendente"
    PAGA = "Paga"
    CANCELADA = "Cancelada"
```

## Passo 3 — Model Cliente

**Conceito:** model = definição da tabela. Herda de `Base`, usa `Column` para declarar colunas.

**Parâmetros de Column:**

- `primary_key=True` → ID único da linha, gerado automaticamente pelo banco
- `index=True` → cria índice, acelera buscas por esse campo
- `nullable=False` → campo obrigatório (equivale a `NOT NULL` no SQL)
- `unique=True` → não aceita valores duplicados (ex: dois clientes com mesmo CPF)



- default= → valor padrão quando não informado

```
# app/models/cliente.py
from sqlalchemy import Column, Integer, String, Enum, Date
from app.database import Base
from app.enums import StatusCliente, TipoPessoa

class Cliente(Base):
    __tablename__ = "clientes"

    id = Column(Integer, primary_key=True, index=True)
    status = Column(Enum(StatusCliente), default=StatusCliente.ATIVO, nullable=False)
    tipo_pessoa = Column(Enum(TipoPessoa), nullable=False)
    nome = Column(String(150), nullable=False)
    cpf_cnpj = Column(String(14), unique=True, nullable=False)
    email = Column(String(100), nullable=False)
    telefone = Column(String(20), nullable=False)
    cidade = Column(String(100), nullable=False)
    uf = Column(String(2), nullable=False)
    data_cadastro = Column(Date, nullable=False)
```

Por que `string(150)` e não só `string` ? SQLite aceita os dois, mas PostgreSQL (produção) exige o tamanho. Já especificar o tamanho é hábito de quem pensa em produção.

Por que `Enum(StatusCliente)` e não `string` ? O banco rejeita valores fora da lista. Se alguém tentar gravar "Atvo" (erro de digitação), o banco rejeita antes de qualquer lógica.

## Passo 4 — Schema Cliente

**Conceito:** schema define o contrato com quem usa a API — o que pode entrar e o que vai sair.

```
# app/schemas/cliente.py
from datetime import date
from pydantic import BaseModel, field_validator
from app.enums import StatusCliente, TipoPessoa
```

```
class ClienteCreate(BaseModel):
    tipo_pessoa : TipoPessoa
    nome : str
    cpf_cnpj : str
    email : str
    telefone : str
    cidade : str
    uf : str
```

```

@field_validator("cpf_cnpj")
@classmethod
def validar_cpf_cnpj(cls, v):
    digitos = "".join(filter(str.isdigit, v))
    if len(digitos) not in (11, 14):
        raise ValueError("CPF deve ter 11 dígitos ou CNPJ 14 dígitos")
    return digitos


class ClienteResponse(BaseModel):
    id : int
    status : StatusCliente
    tipo_pessoa : TipoPessoa
    nome : str
    cpf_cnpj : str
    email : str
    telefone : str
    cidade : str
    uf : str
    data_cadastro : date

    model_config = {"from_attributes": True}

```

**@field\_validator** : executa antes de salvar no banco. Remove pontuação do CPF/CNPJ e valida o tamanho. Se inválido, retorna `422 Unprocessable Entity` com mensagem clara — sem precisar tocar no banco.

**filter(str.isdigit, v)** : percorre cada caractere de `v` e mantém apenas os que são dígitos. `"123.456.789-01"` vira `"12345678901"` .

**model\_config = {"from\_attributes": True}** : permite que o Pydantic leia atributos diretamente de um objeto SQLAlchemy. Sem isso, você precisaria converter manualmente para dicionário antes de retornar.

## Passo 5 — Route Cliente

**Conceito:** route = endpoint da API. O decorator `@router.post("/")` associa a função Python a um método HTTP + caminho URL.

**Analogia VBA:** é como um `Private Sub btnSalvar_Click()` — o gatilho é diferente (HTTP em vez de clique), mas o conceito é o mesmo.

**Códigos HTTP usados:**

- 201 Created → recurso criado com sucesso
- 200 OK → consulta com sucesso

- 404 Not Found → recurso não existe
- 409 Conflict → conflito de dados (CPF duplicado)
- 422 Unprocessable Entity → dado inválido (validação Pydantic)

```
# app/routes/cliente.py
from datetime import date
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from app.database import get_db
from app.models.cliente import Cliente
from app.schemas.cliente import ClienteCreate, ClienteResponse
from app.enums import StatusCliente

router = APIRouter(prefix="/clientes", tags=["Clientes"])

@router.post("/", response_model=ClienteResponse, status_code=201)
def criar_cliente(dados: ClienteCreate, db: Session = Depends(get_db)):
    existente = db.query(Cliente).filter(Cliente.cpf_cnpj == dados.cpf_cnpj).first()
    if existente:
        raise HTTPException(status_code=409, detail="CPF/CNPJ já cadastrado")

    cliente = Cliente(
        **dados.model_dump(),
        status=StatusCliente.ATIVO,
        data_cadastro=date.today()
    )
    db.add(cliente)
    db.commit()
    db.refresh(cliente)
    return cliente

@router.get("/", response_model=list[ClienteResponse])
def listar_clientes(db: Session = Depends(get_db)):
    return db.query(Cliente).all()

@router.get("/{cliente_id}", response_model=ClienteResponse)
def buscar_cliente(cliente_id: int, db: Session = Depends(get_db)):
    cliente = db.query(Cliente).filter(Cliente.id == cliente_id).first()
    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente não encontrado")
    return cliente
```

**\*\*dados.model\_dump()** : converte o schema Pydantic em dicionário Python e "explode" como argumentos nomeados para o construtor. `Cliente(**{"nome": "João", "cpf_cnpj": "123..."})` é equivalente a `Cliente(nome="João", cpf_cnpj="123...")` .

`db.query(Cliente).filter(...).first()` : equivale a `SELECT * FROM clientes WHERE cpf_cnpj = '...' LIMIT 1` . Você escreve Python, o SQLAlchemy traduz para SQL.

`db.refresh(cliente)` : após o `commit` , o objeto em memória pode estar desatualizado (o banco gerou o `id` , por exemplo). O `refresh` sincroniza o objeto com o estado atual do banco.

`Depends(get_db)` : o FastAPI executa `get_db()` antes de chamar a função e injeta o resultado no parâmetro `db` . Você não precisa chamar `get_db()` manualmente.

## Passo 6 — Model Produto

**Conceito novo:** `@property` para campos calculados — equivalente a uma fórmula em célula do Excel.

```
# app/models/produto.py
from sqlalchemy import Column, Integer, String, Enum, Numeric
from sqlalchemy.orm import relationship
from app.database import Base
from app.enums import CategoriaProduto, UnidadeProduto

class Produto(Base):
    __tablename__ = "produtos"

    id = Column(Integer, primary_key=True, index=True)
    codigo = Column(String(20), unique=True, nullable=False)
    nome = Column(String(150), nullable=False)
    categoria = Column(Enum(CategoriaProduto), nullable=False)
    unidade = Column(Enum(UnidadeProduto), nullable=False)
    fabricante = Column(String(100))
    modelo = Column(String(100))
    potencia_w = Column(Numeric(10, 2))
    custo_unitario = Column(Numeric(10, 2), nullable=False)
    preco_venda = Column(Numeric(10, 2), nullable=False)
    qtd_minima = Column(Integer, default=0)

    movimentacoes = relationship("EstoqueMovimentacao", back_populates="produto")

    @property
    def margem_pct(self):
        if not self.preco_venda or self.preco_venda == 0:
            return 0
        return round((self.preco_venda - self.custo_unitario) / self.preco_venda, 4)

    @property
    def qtd_estoque(self):
        return sum(m.quantidade * m.sinal for m in self.movimentacoes)

    @property
```

```
def alerta_estoque(self):
    return self.qtd_estoque < self.qtd_minima
```

`Numeric(10, 2)` : 10 dígitos no total, 2 casas decimais. Mais preciso que `Float` para valores monetários — `Float` pode ter erros de arredondamento.

`qtd_estoque` : acessa `self.movimentacoes` (carregado via `relationship`) e soma `quantidade × sinal`. O saldo nunca é armazenado diretamente — sempre calculado do histórico.

`alerta_estoque` : retorna `True` se o saldo estiver abaixo do mínimo. Usado para destaque visual na interface.

## Passo 7 — Model Venda + ItemVenda

**Conceito novo:** relacionamentos entre tabelas com `ForeignKey` e `relationship()`.

**ForeignKey** → aponta para a chave primária de outra tabela. Equivale ao ID usado em um VLOOKUP no Excel.

**relationship()** → VLOOKUP automático. Após declarar, você acessa `venda.cliente` e `venda.itens` como atributos Python normais — o SQLAlchemy executa os SELECTs necessários.

**back\_populates** → torna o relacionamento bidirecional:

```
Venda diz: "meus itens estão em ItemVenda.venda"
ItemVenda diz: "minha venda está em Venda.itens"
```

```
# app/models/venda.py
from sqlalchemy import Column, Integer, ForeignKey, Date, Numeric, Enum, Text, String
from sqlalchemy.orm import relationship
from app.database import Base
from app.enums import StatusVenda, FormaPagamento

class Venda(Base):
    __tablename__ = "vendas"

    id = Column(Integer, primary_key=True, index=True)
    cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=False)
    id_responsavel = Column(String(6), ForeignKey("colaboradores.id_colaborador"), nullable=False)
    data_orcamento = Column(Date, nullable=False)
    data_aprovacao = Column(Date, nullable=True)
    status_venda = Column(Enum(StatusVenda), default=StatusVenda.ORCAMENTO, nullable=False)
    valor_instalacao = Column(Numeric(10, 2), default=0)
    desconto = Column(Numeric(10, 2), default=0)
    forma_pagamento = Column(Enum(FormaPagamento), nullable=False)
```

```

parcelas          = Column(Integer, default=1)
observacoes       = Column(Text)

cliente          = relationship("Cliente")
responsavel      = relationship("Colaborador")
itens            = relationship("ItemVenda", back_populates="venda")

@property
def valor_equipamentos(self):
    return sum(item.subtotal for item in self.itens)

@property
def valor_final(self):
    return self.valor_equipamentos + float(self.valor_instalacao or 0) - float(self.descon

# app/models/item_venda.py
from sqlalchemy import Column, Integer, ForeignKey, Numeric
from sqlalchemy.orm import relationship
from app.database import Base

class ItemVenda(Base):
    __tablename__ = "itens_venda"

    id              = Column(Integer, primary_key=True, index=True)
    venda_id        = Column(Integer, ForeignKey("vendas.id"), nullable=False)
    produto_id      = Column(Integer, ForeignKey("produtos.id"), nullable=False)
    quantidade      = Column(Numeric(10, 3), nullable=False)
    preco_unitario  = Column(Numeric(10, 2), nullable=False)

    venda          = relationship("Venda", back_populates="itens")
    produto         = relationship("Produto")

    @property
    def subtotal(self):
        return float(self.quantidade) * float(self.preco_unitario)

```

Por que `relationship()` usa strings ( "Cliente" ) em vez da classe diretamente? Porque evita importações circulares. O SQLAlchemy resolve os nomes em tempo de execução, quando todas as classes já foram carregadas.

## Passo 8 — Schema e Route Venda

**Conceito novo:** schema aninhado — uma lista de itens dentro do schema da venda.

```

# app/schemas/venda.py
from datetime import date

```

```
from pydantic import BaseModel, field_validator
from typing import Optional
from app.enums import StatusVenda, FormaPagamento


class ItemVendaCreate(BaseModel):
    produto_id : int
    quantidade : float
    preco_unitario : Optional[float] = None # None = usa preço do catálogo

    @field_validator("quantidade")
    @classmethod
    def validar_quantidade(cls, v):
        if v <= 0:
            raise ValueError("Quantidade deve ser maior que zero")
        return v


class ItemVendaResponse(BaseModel):
    id : int
    produto_id : int
    quantidade : float
    preco_unitario : float
    subtotal : float

    model_config = {"from_attributes": True}


class VendaCreate(BaseModel):
    cliente_id : int
    id_responsavel : Optional[str] = None
    valor_instalacao : float = 0
    desconto : float = 0
    forma_pagamento : FormaPagamento
    parcelas : int = 1
    observacoes : Optional[str] = None
    itens : list[ItemVendaCreate]


class VendaResponse(BaseModel):
    id : int
    cliente_id : int
    id_responsavel : Optional[str]
    data_orcamento : date
    data_aprovacao : Optional[date]
    status_venda : StatusVenda
    valor_instalacao : float
    desconto : float
    forma_pagamento : FormaPagamento
    parcelas : int
    itens : list[ItemVendaResponse]
    valor_equipamentos : float
    valor_final : float
```

```
model_config = {"from_attributes": True}
```

## Route de criação (trecho principal):

```
# app/routes/venda.py – função criar_venda
@router.post("/", response_model=VendaResponse, status_code=201)
def criar_venda(dados: VendaCreate, db: Session = Depends(get_db)):
    cliente = db.query(Cliente).filter(Cliente.id == dados.cliente_id).first()
    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente não encontrado")

    if not dados.itens:
        raise HTTPException(status_code=422, detail="A venda deve ter pelo menos um item")

    venda = Venda(
        cliente_id      = dados.cliente_id,
        id_responsavel  = dados.id_responsavel,
        data_orcamento = date.today(),
        status_venda    = StatusVenda.ORCAMENTO,
        valor_instalacao = dados.valor_instalacao,
        desconto        = dados.desconto,
        forma_pagamento = dados.forma_pagamento,
        parcelas        = dados.parcelas,
        observacoes     = dados.observacoes,
    )
    db.add(venda)
    db.flush() # gera venda.id sem fechar a transação

    for item_data in dados.itens:
        produto = db.query(Produto).filter(Produto.id == item_data.produto_id).first()
        if not produto:
            raise HTTPException(status_code=404,
                                detail=f"Produto id={item_data.produto_id} não encontrado")

        # Se não informou preço, usa o do catálogo
        preco = item_data.preco_unitario if item_data.preco_unitario else float(produto.preco_

    item = ItemVenda(
        venda_id      = venda.id,
        produto_id    = item_data.produto_id,
        quantidade     = item_data.quantidade,
        preco_unitario = preco,
    )
    db.add(item)

    db.commit()
    db.refresh(venda)
    return venda
```



**Atenção:** após adicionar um campo novo ao schema, sempre verificar se a route que constrói o objeto também inclui esse campo. Erros desse tipo não causam exceção — o campo simplesmente fica `null` silenciosamente.

## Passo 9 — Model Colaborador

**Conceito novo:** PK string gerada pelo sistema (não pelo banco).

O banco gera IDs inteiros automáticos ( 1, 2, 3... ). Para IDs legíveis como `COL001`, o sistema gera o valor antes de inserir.

```
# app/models/colaborador.py
from sqlalchemy import Column, String, Enum, Numeric, Date, Text
from app.database import Base
from app.enums import TipoContrato, StatusColaborador

class Colaborador(Base):
    __tablename__ = "colaboradores"

    id_colaborador = Column(String(6), primary_key=True)
    nome = Column(String(150), nullable=False)
    cpf = Column(String(11), unique=True, nullable=False)
    cargo = Column(String(100))
    tipo_contrato = Column(Enum(TipoContrato), nullable=False)
    percentual_comissao = Column(Numeric(5, 4), nullable=False) # 0.0600 = 6%
    telefone = Column(String(20))
    email = Column(String(100))
    data_admissao = Column(Date, nullable=False)
    status = Column(Enum(StatusColaborador), default=StatusColaborador.ATIVO)
    observacoes = Column(Text)
```

**Geração do ID (na route):**

```
def gerar_id_colaborador(db: Session) -> str:
    ultimo = (
        db.query(Colaborador)
        .order_by(Colaborador.id_colaborador.desc())
        .first()
    )
    numero = int(ultimo.id_colaborador[3:]) + 1 if ultimo else 1
    return f"COL{numero:03d}"
```

**Leitura passo a passo:**

```

"COL007"[3:]    → "007"          (fatia a partir do índice 3)
int("007")      → 7              (converte para inteiro)
7 + 1           → 8              (incrementa)
f"COL{8:03d}"   → "COL008"      (:03d = 3 dígitos com zeros à esquerda)

```

## Passo 10 — Model EstoqueMovimentacao

**Conceito:** ledger imutável. Nunca armazene o saldo diretamente — sempre calcule pela soma das movimentações.

**Por que?** Histórico auditável, impossível de "sumir" dados por acidente, qualquer erro é corrigido com um estorno (nova linha), não editando registros existentes. Mesmo princípio da contabilidade.

```

# app/models/estoque.py
from datetime import datetime
from sqlalchemy import Column, Integer, String, ForeignKey, Numeric, Enum, DateTime, Text
from sqlalchemy.orm import relationship
from app.database import Base
from app.enums import TipoMovimento

```

```

TIPOS_POSITIVOS = {TipoMovimento.ENTRADA, TipoMovimento.AJUSTE_POSITIVO}

```

```

class EstoqueMovimentacao(Base):
    __tablename__ = "estoque_movimentacoes"

    id = Column(Integer, primary_key=True, index=True)
    data = Column(DateTime, default=datetime.now, nullable=False)
    tipo_mov = Column(Enum(TipoMovimento), nullable=False)
    id_produto = Column(Integer, ForeignKey("produtos.id"), nullable=False)
    quantidade = Column(Numeric(10, 3), nullable=False)
    valor_unitario = Column(Numeric(10, 2), nullable=False)
    id_venda = Column(Integer, ForeignKey("vendas.id"), nullable=True)
    responsavel = Column(String(150))
    observacoes = Column(Text)

    produto = relationship("Produto", back_populates="movimentacoes")
    venda = relationship("Venda")

    @property
    def sinal(self):
        return 1 if self.tipo_mov in TIPOS_POSITIVOS else -1

    @property
    def valor_total(self):
        return float(self.quantidade) * float(self.valor_unitario)

```

O `sinal` resolve o cálculo do saldo:

| Tipo    | Sinal | Efeito          |
|---------|-------|-----------------|
| Entrada | +1    | Aumenta estoque |
| Saída   | -1    | Diminui estoque |
| Ajuste+ | +1    | Aumenta estoque |
| Ajuste- | -1    | Diminui estoque |

`qtd_estoque = SUM(quantidade × sinal)` — no model `Produto`:

```
@property
def qtd_estoque(self):
    return sum(m.quantidade * m.sinal for m in self.movimentacoes)
```

## Passo 11 — Model Financeiro

Cada linha é um lançamento financeiro — receita ou despesa.

```
# app/models/financeiro.py
from sqlalchemy import Column, Integer, ForeignKey, Numeric, Enum, Date, String, Text
from sqlalchemy.orm import relationship
from app.database import Base
from app.enums import TipoLancamento, CategoriaFinanceiro, StatusPagamento, FormaPagamento

class Financeiro(Base):
    __tablename__ = "financeiro"

    id = Column(Integer, primary_key=True, index=True)
    tipo = Column(Enum(TipoLancamento), nullable=False)
    categoria = Column(Enum(CategoriaFinanceiro), nullable=False)
    descricao = Column(String(200), nullable=False)
    valor = Column(Numeric(10, 2), nullable=False)
    data_vencimento = Column(Date, nullable=False)
    data_pagamento = Column(Date, nullable=True)
    status_pagamento = Column(Enum(StatusPagamento), default=StatusPagamento.PENDENTE)
    forma_pagamento = Column(Enum(FormaPagamento), nullable=False)
    id_venda = Column(Integer, ForeignKey("vendas.id"), nullable=True)
    observacoes = Column(Text)

    venda = relationship("Venda")
```

Route de pagamento (PATCH):

```
@router.patch("/{lancamento_id}/pagar", response_model=LancamentoResponse)
def pagar_lancamento(lancamento_id: int, db: Session = Depends(get_db)):
    lancamento = db.query(Financeiro).filter(Financeiro.id == lancamento_id).first()
    if not lancamento:
        raise HTTPException(status_code=404, detail="Lançamento não encontrado")
    if lancamento.status_pagamento == StatusPagamento.PAGO:
        raise HTTPException(status_code=409, detail="Lançamento já está pago")

    lancamento.status_pagamento = StatusPagamento.PAGO
    lancamento.data_pagamento = date.today()
    db.commit()
    db.refresh(lancamento)
    return lancamento
```

### Route de saldo:

```
@router.get("/saldo")
def saldo_financeiro(db: Session = Depends(get_db)):
    lancamentos = db.query(Financeiro).filter(
        Financeiro.status_pagamento == StatusPagamento.PAGO
    ).all()
    receitas = sum(float(l.valor) for l in lancamentos if l.tipo == TipoLancamento.RECEITA)
    despesas = sum(float(l.valor) for l in lancamentos if l.tipo == TipoLancamento.DESPESA)
    return {"receitas_pagas": receitas, "despesas_pagas": despesas, "saldo": receitas - despesas}
```

## Passo 12 — Aprovação Completa

A aprovação é uma **transação atômica**: 5 operações na mesma sessão. Se qualquer uma falhar, todas são revertidas.

```
@router.patch("/{venda_id}/aprovar", response_model=VendaResponse)
def aprovar_venda(venda_id: int, db: Session = Depends(get_db)):
    venda = db.query(Venda).filter(Venda.id == venda_id).first()
    if not venda:
        raise HTTPException(status_code=404, detail="Venda não encontrada")
    if venda.status_venda != StatusVenda.ORCAMENTO:
        raise HTTPException(status_code=409,
            detail=f"Venda com status '{venda.status_venda.value}' não pode ser aprovada")

    # ETAPA 1: validar estoque antes de qualquer baixa
    for item in venda.itens:
        produto = db.query(Produto).filter(Produto.id == item.produto_id).first()
        if produto.qtd_estoque < float(item.quantidade):
            raise HTTPException(status_code=422,
                detail=f"Estoque insuficiente para '{produto.nome}'. "
```

```
f"Disponível: {produto.qtd_estoque}, solicitado: {float(item.quantidade
```

```
# ETAPA 2: muda status
venda.status_venda = StatusVenda.APROVADO
venda.data_aprovacao = date.today()

# ETAPA 3: baixa de estoque
for item in venda.itens:
    db.add(EstoqueMovimentacao(
        tipo_mov = TipoMovimento.SAIDA,
        id_produto = item.produto_id,
        quantidade = item.quantidade,
        valor_unitario = item.preco_unitario,
        id_venda = venda.id,
        responsavel = "Sistema",
        observacoes = f"Baixa automática - Venda #{venda.id}",
    ))

# ETAPA 4: lançamentos financeiros pendentes
db.add(Financeiro(
    tipo=TipoLancamento.RECEITA, categoria=CategoriaFinanceiro.VENDA_EQUIPAMENTOS,
    descricao=f"Equipamentos - Venda #{venda.id}", valor=venda.valor_equipamentos,
    data_vencimento=date.today(), status_pagamento=StatusPagamento.PENDENTE,
    forma_pagamento=venda.forma_pagamento, id_venda=venda.id,
))
if float(venda.valor_instalacao or 0) > 0:
    db.add(Financeiro(
        tipo=TipoLancamento.RECEITA, categoria=CategoriaFinanceiro.SERVICO_INSTALACAO,
        descricao=f"Instalação - Venda #{venda.id}", valor=venda.valor_instalacao,
        data_vencimento=date.today(), status_pagamento=StatusPagamento.PENDENTE,
        forma_pagamento=venda.forma_pagamento, id_venda=venda.id,
    ))

# ETAPA 5: prestação de contas (se houver responsável)
if venda.id_responsavel:
    colaborador = db.query(Colaborador).filter(
        Colaborador.id_colaborador == venda.id_responsavel
    ).first()
    ultima = db.query(PrestacaoContas).order_by(PrestacaoContas.id_prestacao.desc()).first
    numero = int(ultima.id_prestacao[2:]) + 1 if ultima else 1

    db.add(PrestacaoContas(
        id_prestacao = f"PC{numero:04d}",
        data = date.today(),
        id_venda = venda.id,
        id_colaborador = venda.id_responsavel,
        valor_recebido = venda.valor_equipamentos,
        custo_instalacao = venda.valor_instalacao or 0,
        percentual_comissao = colaborador.percentual_comissao,
        status_pagto = StatusPrestacao.PENDENTE,
    ))

db.commit() # ← único commit, tudo junto
```

```
db.refresh(venda)
return venda
```

## Passo 13 — Model PrestacaoContas

```
# app/models/prestacao_contas.py
from sqlalchemy import Column, String, Integer, ForeignKey, Numeric, Date, Enum, Text
from sqlalchemy.orm import relationship
from app.database import Base
from app.enums import StatusPrestacao

class PrestacaoContas(Base):
    __tablename__ = "prestacao_contas"

    id_prestacao = Column(String(7), primary_key=True)
    data = Column(Date, nullable=False)
    id_venda = Column(Integer, ForeignKey("vendas.id"), nullable=False)
    id_colaborador = Column(String(6), ForeignKey("colaboradores.id_colaborador"), nullable=False)
    valor_recebido = Column(Numeric(10, 2), nullable=False)
    custo_instalacao = Column(Numeric(10, 2), default=0)
    percentual_comissao = Column(Numeric(5, 4), nullable=False)
    status_pagto = Column(Enum(StatusPrestacao), default=StatusPrestacao.PENDENTE)
    data_pagto = Column(Date, nullable=True)
    observacoes = Column(Text)

    venda = relationship("Venda")
    colaborador = relationship("Colaborador")

    @property
    def valor_comissao(self):
        return round(float(self.valor_recebido) * float(self.percentual_comissao), 2)

    @property
    def valor_liquido(self):
        return float(self.valor_recebido) - self.valor_comissao
```

**Route de pagamento de comissão** — pagar a comissão gera automaticamente uma despesa financeira:

```
@router.patch("/{prestacao_id}/pagar", response_model=PrestacaoResponse)
def pagar_prestacao(prestacao_id: str, db: Session = Depends(get_db)):
    prestacao = db.query(PrestacaoContas).filter(
        PrestacaoContas.id_prestacao == prestacao_id
    ).first()
    if not prestacao:
```

```

        raise HTTPException(status_code=404, detail="Prestação não encontrada")
    if prestacao.status_pagto == StatusPrestacao.PAGA:
        raise HTTPException(status_code=409, detail="Comissão já foi paga")

    prestacao.status_pagto = StatusPrestacao.PAGA
    prestacao.data_pagto = date.today()

    db.add(Financeiro(
        tipo = TipoLancamento.DESPESA,
        categoria = CategoriaFinanceiro.COMISSAO,
        descricao = f"Comissão {prestacao.id_colaborador} - Venda #{prestacao.id_venda}"
        valor = prestacao.valor_comissao,
        data_vencimento = date.today(),
        data_pagamento = date.today(),
        status_pagamento = StatusPagamento.PAGO,
        forma_pagamento = FormaPagamento.PIX,
        id_venda = prestacao.id_venda,
    ))

    db.commit()
    db.refresh(prestacao)
    return prestacao

```

## Passo 14 — main.py final

---

```

from fastapi import FastAPI
from app.database import Base, engine
from app.models import cliente as _c
from app.models import produto as _p
from app.models import venda as _v
from app.models import item_venda as _i
from app.models import colaborador as _col
from app.models import estoque as _e
from app.models import financeiro as _f
from app.models import prestacao_contas as _pc
from app.routes import (
    cliente, produto, venda, colaborador,
    estoque, financeiro, prestacao_contas, dashboard
)

Base.metadata.create_all(bind=engine)

app = FastAPI(title="Sistema Solar", version="1.0")

app.include_router(cliente.router)
app.include_router(produto.router)
app.include_router(venda.router)
app.include_router(colaborador.router)
app.include_router(estoque.router)

```

```
app.include_router(financieiro.router)
app.include_router(prestacao_contas.router)
app.include_router/dashboard.router)
```

## Por que importar os models com as \_x ?

O SQLAlchemy só sabe que uma tabela existe se o model foi **carregado em memória** antes de `create_all`. Importar o módulo é suficiente. O `as _x` é uma convenção que diz: *"importo pelo efeito colateral, não vou usar a variável"*.

## Passo 15 — Dashboard endpoint

---

```
# app/routes/dashboard.py
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from app.database import get_db
from app.models.venda import Venda
from app.models.produto import Produto
from app.models.financieiro import Financeiro
from app.enums import StatusVenda, StatusPagamento, TipoLancamento

router = APIRouter(prefix="/dashboard", tags=["Dashboard"])

@router.get("/")
def get_dashboard(db: Session = Depends(get_db)):
    todas_vendas = db.query(Venda).all()
    vendas_aprovadas = [v for v in todas_vendas if v.status_venda == StatusVenda.APROVADO]
    vendas_pipeline = [v for v in todas_vendas
                       if v.status_venda in (StatusVenda.ORCAMENTO, StatusVenda.NEGOCIACAO)]

    receita_aprovada = sum(v.valor_final for v in vendas_aprovadas)
    pipeline = sum(v.valor_final for v in vendas_pipeline)
    ticket_medio = receita_aprovada / len(vendas_aprovadas) if vendas_aprovadas else 0

    lancamentos_pagos = db.query(Financeiro).filter(
        Financeiro.status_pagamento == StatusPagamento.PAGO
    ).all()
    receitas_pagas = sum(float(l.valor) for l in lancamentos_pagos if l.tipo == TipoLancamento)
    despesas_pagas = sum(float(l.valor) for l in lancamentos_pagos if l.tipo == TipoLancamento)

    produtos = db.query(Produto).all()
    itens_em_alerta = sum(1 for p in produtos if p.alerta_estoque)
    valor_estoque = sum(float(p.custo_unitario) * float(p.qtd_estoque) for p in produtos)

    return {
        "total_vendas" : len(todas_vendas),
        "receita_aprovada": round(receita_aprovada, 2),
        "pipeline" : round(pipeline, 2),
```



```

    "saldo_financeiro": round(receitas_pagas - despesas_pagas, 2),
    "ticket_medio"      : round(ticket_medio, 2),
    "itens_em_alerta"   : itens_em_alerta,
    "valor_em_estoque"  : round(valor_estoque, 2),
}

```

## Passo 16 — Frontend Streamlit

### Arquitetura

```

Usuário (browser :8501)
  ↓ interação
Streamlit (porta 8501)
  ↓ chamadas HTTP via requests
FastAPI (porta 8000)
  ↓ queries SQLAlchemy
SQLite (solar.db)

```

O Streamlit **não acessa o banco diretamente** — ele consome a API como qualquer frontend (React, mobile, etc.) faria.

### Como rodar (dois terminais simultâneos)

```

# Terminal 1 – backend
uvicorn app.main:app --reload

# Terminal 2 – frontend (dentro de streamlit_app/)
cd streamlit_app
streamlit run app.py

```

### api\_client.py — camada de comunicação

Centraliza todas as chamadas HTTP. Nenhuma tela faz `requests` diretamente.

```

import requests

BASE_URL = "http://127.0.0.1:8000"

def get_dashboard():
    return requests.get(f"{BASE_URL}/dashboard/").json()
def listar_clientes():
    return requests.get(f"{BASE_URL}/clientes/").json()
def criar_cliente(d):
    return requests.post(f"{BASE_URL}/clientes/", json=d)
def listar_produtos():
    return requests.get(f"{BASE_URL}/produtos/").json()
def criar_produto(d):
    return requests.post(f"{BASE_URL}/produtos/", json=d)
def listar_colaboradores():
    return requests.get(f"{BASE_URL}/colaboradores/").json()

```

```

def listar_vendas(): return requests.get(f"{BASE_URL}/vendas/").json()
def criar_venda(d): return requests.post(f"{BASE_URL}/vendas/", json=d)
def aprovar_venda(vid): return requests.patch(f"{BASE_URL}/vendas/{vid}/aprovar")
def listar_lancamentos(): return requests.get(f"{BASE_URL}/financeiro/").json()
def pagar_lancamento(lid): return requests.patch(f"{BASE_URL}/financeiro/{lid}/pagar")
def listar_prestacoes(): return requests.get(f"{BASE_URL}/prestacoes/").json()
def pagar_prestacao(pid): return requests.patch(f"{BASE_URL}/prestacoes/{pid}/pagar")
def registrar_movimentacao(d): return requests.post(f"{BASE_URL}/estoque/movimentacoes", json=
def saldo_produto(pid): return requests.get(f"{BASE_URL}/estoque/produtos/{pid}/saldo").jso

```

## Conceitos Streamlit

**st.session\_state** — memória que persiste entre reruns. O Streamlit reexecuta o arquivo inteiro a cada interação. Sem `session_state`, dados como a lista de itens da venda sumiriam a cada clique.

```

if "itens_venda" not in st.session_state:
    st.session_state.itens_venda = []

if st.button("Adicionar"):
    st.session_state.itens_venda.append(novo_item)
    st.rerun() # força reexecução para atualizar a lista na tela

```

**st.form()** — agrupa campos e só dispara a lógica quando o usuário clica em "Enviar". Evita que o script rode a cada letra digitada.

**st.tabs()** — cria abas dentro de uma página. Padrão usado em todas as telas: aba "Lista" + aba "Novo Cadastro".

**st.columns()** — divide a linha em colunas. `col1, col2 = st.columns(2)` cria duas colunas iguais.

**st.metric()** — exibe um KPI visual com título e valor. Usado no Dashboard.

**st.dataframe()** — exibe um DataFrame pandas como tabela interativa.

## app.py — Dashboard

```

import streamlit as st
import api_client

st.set_page_config(page_title="Sistema Solar", page_icon="☀️", layout="wide")
st.title("☀️ Sistema Solar")
st.markdown("---")

try:
    dados = api_client.get_dashboard()
except Exception:
    st.error("API offline. Verifique se o servidor FastAPI está rodando.")
    st.stop()

```

```
col1, col2, col3, col4 = st.columns(4)
col1.metric("Total de Vendas", dados["total_vendas"])
col2.metric("Receita Aprovada", f"R$ {dados['receita_aprovada']:, .2f}")
col3.metric("Pipeline", f"R$ {dados['pipeline']:, .2f}")
col4.metric("Saldo Financeiro", f"R$ {dados['saldo_financeiro']:, .2f}")

st.markdown("---")
col5, col6, col7 = st.columns(3)
col5.metric("Ticket Médio", f"R$ {dados['ticket_medio']:, .2f}")
col7.metric("Valor Estoque", f"R$ {dados['valor_em_estoque']:, .2f}")
with col6:
    alerta = dados["itens_em_alerta"]
    st.metric("⚠️ Itens em Alerta" if alerta > 0 else "✅ Itens em Alerta", alerta)
```

## Erros Comuns

| Erro                                    | Causa                                                                                        | Solução                                                                                    |
|-----------------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| KeyError:<br><TipoPessoa.PF>            | Enum definido em dois arquivos diferentes — Python trata como tipos distintos                | Centralizar todos os enums em <code>enums.py</code>                                        |
| Table already defined                   | Model SQLAlchemy criado dentro da pasta <code>schemas/</code>                                | Models sempre em <code>models/</code> , schemas em <code>schemas/</code>                   |
| cannot import name 'X' from 'app.enums' | Arquivo <code>__pycache__</code> com versão antiga do módulo                                 | Parar o uvicorn (Ctrl+C) e reiniciar do zero                                               |
| Campo retorna <code>0</code>            | Swagger preenche campos numéricos com <code>0</code> por padrão — <code>0 is not None</code> | Usar <code>Optional[float] = None</code> e checar com <code>if valor</code>                |
| Nova coluna não aparece no banco        | <code>create_all</code> só cria tabelas novas, não altera existentes                         | Deletar <code>solar.db</code> em desenvolvimento                                           |
| Campo novo salvo como <code>null</code> | Route constrói o objeto campo a campo e o novo campo foi esquecido                           | Incluir o campo explicitamente na construção do objeto                                     |
| Estoque fica negativo                   | Aprovação não validava saldo antes de baixar                                                 | Verificar <code>qtd_estoque &gt;= quantidade</code> antes do loop de baixa                 |
| Table 'X' is already defined            | Mesmo model registrado duas vezes (import duplicado ou herança dupla de Base)                | Verificar imports em <code>main.py</code> , garantir que cada model é importado uma só vez |

| Erro                                   | Causa                                                      | Solução                                   |
|----------------------------------------|------------------------------------------------------------|-------------------------------------------|
| ImportError: cannot import name 'Base' | from app.database import Base antes de database.py existir | Criar database.py antes de qualquer model |

## Próximos Passos

### Telas Streamlit pendentes

- [ ] 04\_Financeiro.py — listar lançamentos, registrar pagamento, ver saldo
- [ ] 05\_Prestacoes.py — listar prestações, pagar comissão

## Passo 17 — Módulo de Projetos de Homologação

Rastreia o processo de aprovação do projeto fotovoltaico junto à distribuidora de energia após a instalação.

### Campos principais

| Campo           | Tipo    | Descrição                                                         |
|-----------------|---------|-------------------------------------------------------------------|
| cliente_id      | FK      | Cliente vinculado                                                 |
| uc              | String  | Unidade Consumidora (número do medidor)                           |
| data_entrada    | Date    | Data de submissão à distribuidora                                 |
| data_resultado  | Date    | Prazo esperado de aprovação                                       |
| protocolo       | String  | Número de protocolo na distribuidora                              |
| status_projeto  | Enum    | Em Análise / Aprovado / Aprovado c/ Obras / Falta TRT / Cancelado |
| status_vistoria | Enum    | Não Solicitado / Solicitado / Realizado                           |
| kwp             | Numeric | Potência do sistema                                               |
| inversor        | String  | Modelo do inversor                                                |

### Importação do histórico

O script `importar_projetos.py` na raiz do projeto importa dados do Excel. Execute uma única vez:

```
.\.venv\Scripts\python.exe importar_projetos.py
```

O script normaliza nomes de clientes (remove anotações entre parênteses), cria os clientes no banco e vincula os projetos.

## Comportamento na tela

- Lista mostra alertas em vermelho para projetos com prazo vencido
- Filtros por status e status de vistoria

## Passo 18 — Módulo de Ordens de Serviço

Gerencia serviços de manutenção prestados aos clientes. Ao concluir uma OS com custo, **gera automaticamente um lançamento de Receita no Financeiro**.

### Campos principais

| Campo            | Tipo      | Descrição                                                       |
|------------------|-----------|-----------------------------------------------------------------|
| id_os            | String PK | OS0001, OS0002... (gerado automaticamente)                      |
| cliente_id       | FK        | Cliente atendido                                                |
| id_tecnico       | FK        | Colaborador responsável                                         |
| tipo_servico     | Enum      | Preventiva / Corretiva / Limpeza / Inspeção / Garantia / Outros |
| descricao        | Text      | Descrição do problema ou serviço                                |
| status_os        | Enum      | Aberta / Em Andamento / Concluída / Cancelada                   |
| data_abertura    | Date      | Quando a OS foi aberta                                          |
| data_agendamento | Date      | Quando o atendimento foi agendado                               |
| data_conclusao   | Date      | Preenchida automaticamente ao concluir                          |
| forma_pagamento  | Enum      | Como o cliente paga                                             |
| custo_total      | Numeric   | Valor cobrado pelo serviço                                      |

### Regra de negócio

```
# Em routes/ordem_servico.py – ao atualizar status para Concluída:  
concluindo_ agora = (  
    novo_status == StatusOS.CONCLUIDA
```

```
    and status_anterior is not StatusOS.CONCLUIDA
    and os_.tipo_servico is not TipoServico.GARANTIA # Garantia é gratuita
    and float(os_.custo_total or 0) > 0
)
if concluindo_agora:
    db.add(Financeiro(
        tipo=TipoLancamento.RECEITA,
        categoria=CategoriaFinanceiro.MANUTENCAO,
        descricao=f"OS {os_.id_os} - {os_.tipo_servico.value}",
        valor=os_.custo_total,
        ...
    ))
```

## Para produção

- **Alembic** — ferramenta de migração de banco. Quando você adiciona uma coluna ao model, o Alembic gera um script SQL para alterar a tabela sem perder dados. Substitui o "deletar o solar.db" de desenvolvimento.
- **PostgreSQL** — substitui SQLite para múltiplos usuários simultâneos. SQLite trava para escrita concorrente.
- **Deploy Railway/Render** — plataformas que hospedam a aplicação online. O usuário acessa via URL pública, sem precisar rodar localmente.

## O que já está implementado

- ☒ Cadastro completo de Clientes, Produtos, Colaboradores
- ☒ Filtros nas listas (por status, tipo, data)
- ☒ Editar e excluir em todas as entidades
- ☒ Vendas com aprovação, baixa de estoque e lançamento financeiro automático
- ☒ Controle de Estoque com alertas de estoque mínimo
- ☒ Financeiro com receitas, despesas e saldo
- ☒ Prestações de contas e comissões de colaboradores
- ☒ Projetos de Homologação com importação de histórico Excel
- ☒ Ordens de Serviço com geração automática de receita ao concluir
- ☒ Script de inicialização ( `iniciar.bat` ) para uso no cliente

## Próximos passos sugeridos

- Relatórios em PDF (resumo de vendas, OS por período, extrato financeiro)
- Autenticação com login/senha
- Alembic para migrações seguras de banco de dados
- Deploy online (Railway ou Render) com PostgreSQL